

UT3.- ESTRUCTURAS DE CONTROL, ARRAYS Y

FORMULARIOS

RESULTADOS DE APRENDIZAJE ASOCIADOS
3. Escribe bloques de sentencias embebidos en lenguajes de marcas, seleccionando y utilizando las estructuras de programación.
CRITERIOS DE EVALUACIÓN
a) Se han utilizado mecanismos de decisión en la creación de bloques de sentencias. b) Se han utilizado bucles y se ha verificado su funcionamiento. c) Se han utilizado “arrays” para almacenar y recuperar conjuntos de datos. d) Se han creado y utilizado funciones. e) Se han utilizado formularios web para interactuar con el usuario del navegador web. f) Se han empleado métodos para recuperar la información introducida en el formulario. g) Se han añadido comentarios al código. h) Se han utilizado herramientas y entornos para facilitar la programación, prueba y depuración del código.

UT3.- ESTRUCTURAS DE CONTROL, ARRAYS Y

FORMULARIOS

Índice de contenido

1.- Estructura de los contenidos.....	3
2.- Estructuras de control	3
2.1.- Estructuras condicionales	3
2.2.- Estructuras de repetición	8
2.2.1.- Bucle for.....	8
2.2.2.- Bucle <i>while</i>	10
2.2.3.- Bucle <i>do-while</i>	11
2.2.4.- Otras estructuras de control	14
3.- Arrays.....	18
3.1.- Operadores y arrays.....	26
3.1.1.- Operador Igual y operador Idéntico	26
3.1.1.- Operador +.....	28
4.- Funciones y librerías.....	31
4.1.- Funciones predefinidas	31
4.2.- Funciones definidas por el usuario	33
4.3.- Paso de parámetros por copia y por referencia.....	36
4.4.- Funciones como argumentos	37
5.- Excepciones y errores.....	40
5.1.- Errores	40
5.2.- Excepciones	42
5.3.- Excepciones error	44
6.- Formularios.....	45
6.1.- Formulario de <i>login</i>	49
6.2.- Formulario y procesamiento en un solo fichero	51
7.- Glosario de términos.....	53
8.- Referencias bibliográficas	53

1.- Estructura de los contenidos

En este tema profundizamos un poco más en el conocimiento del lenguaje PHP. Aprenderemos a utilizar las estructuras de control, arrays y los formularios.

2.- Estructuras de control



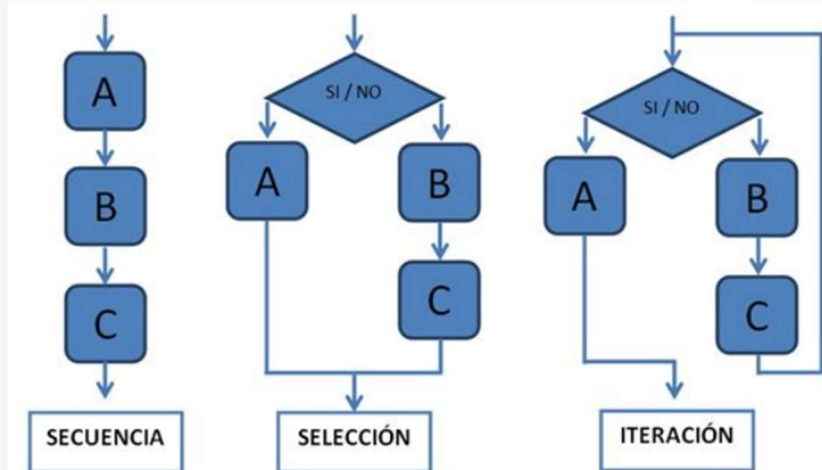
Antes de comenzar, debemos preguntarnos lo siguiente:

¿Qué es el flujo de un documento PHP?

PHP cuenta con las estructuras de control habituales en la programación estructurada para realizar sentencias condicionales o de selección y de repetición. La sintaxis es muy parecida a la de Java o C.

- Selección

- Iteración



2.1.- Estructuras condicionales



Manual:

<https://www.php.net/manual/es/control-structures.if.php>

Las estructuras condicionales de PHP son:

- if
- if-else
- if-elseif
- switch

La sintaxis general es:

```
if(condición)
    instrucción;
```



Si se cumple la condición, se ejecuta la instrucción. Si no se cumple, no se ejecuta.

Para agrupar dentro del *if* más de una sentencia, se encierran entre llaves.

```
if(condición)
{
    instrucción1;
    instrucción2;
}
```



La condición es una expresión que se evalúa a verdadero o falso, siguiendo las normas de conversión a *boolean* si es necesario.

Ejemplo:

```
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8">
    <title></title>
  </head>
  <body>
    <?php
      $var = 3;
      if($var < 0)
      {
        echo "Es menor que cero";
      }else{
        echo "Es mayor o igual que cero";
      }
    ?>
  </body>
</html>
```

Si lo vemos en el navegador:

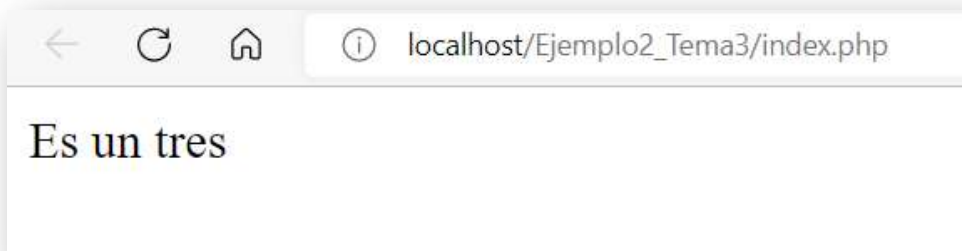


Si se anidan varias sentencias condicionales, se puede usar *elseif*, que es equivalente a *else if*.

Ejemplo:

```
<?php
    $var = 3;
    if($var == 1)
    {
        echo "Es un uno";
    }elseif ($var == 2){
        echo "Es un dos";
    }elseif($var == 3){
        echo "Es un tres";
    }else{
        echo "No es un uno, ni un dos, ni un tres";
    }
?>
```

La salida será, por tanto:



¡CUIDADO! ERROR COMÚN

```
if($numero=1)
{
}

```

¿Realmente qué estamos haciendo en el código anterior?

**Manual:**<https://www.php.net/manual/es/control-structures.switch.php>

Para agrupar varios *if* puede ser útil la estructura ***switch***, también habitual en otros lenguajes. El ejemplo siguiente es equivalente al ejemplo anterior:

```
<?php
    $var = 3;
    switch($var)
    {
        case 1:
            echo "Es un 1";
            break;

        case 2:
            echo "Es un 2";
            break;

        case 3:
            echo "Es un 3";
            break;
        default:
            echo "No es un 1, ni un 2, ni un 3";
    }
?>
```

Según el valor de *\$var*, se ejecutará un *case* u otro. La sección *default* se ejecuta cuando no se da ninguno de los *case*



Si no hay un **break** al final de un *case*, la ejecución continuará con el siguiente. ¡Pruébalo y verás lo que ocurre!

2.2.- Estructuras de repetición

Las estructuras de repetición o bucles sirven para repetir un conjunto de instrucciones mientras se dé una condición.

PHP cuenta con las estructuras habituales:

- for
- while
- do-while

que tienen la misma sintaxis que en Java o C.

2.2.1.- Bucle for



Manual:

<https://www.php.net/manual/es/control-structures.for.php>

Para el bucle *for*, la sintaxis es:

```
for(instrucciones de inicialización; condición; instrucciones de iteración) {  
    instrucciones del bucle;  
}
```



Las instrucciones de inicialización se realizan una única vez al llegar al bucle.

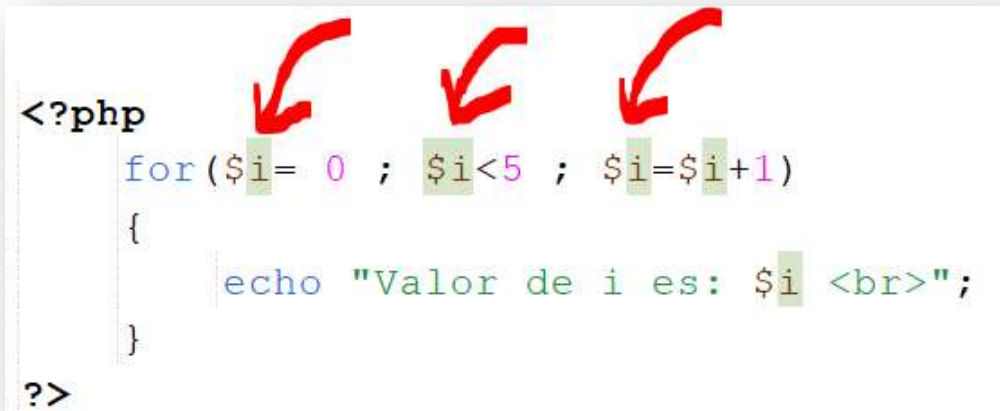


Las instrucciones dentro del cuerpo del bucle se repetirán mientras se cumpla la condición.

Después de ejecutar las instrucciones de inicialización se evalúa la condición. Si se cumple, se ejecutan las instrucciones del bucle y las

instrucciones de iteración. Después se vuelve a comprobar la condición. El proceso se repite hasta que la condición deja de cumplirse y a partir de ahí la ejecución continúa con las instrucciones que estés después de dicho bucle.

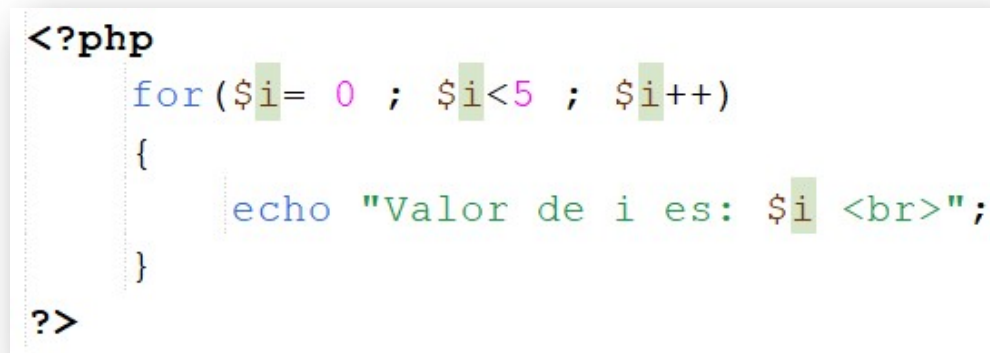
Ejemplo:



```
<?php
    for($i= 0 ; $i<5 ; $i=$i+1)
    {
        echo "Valor de i es: $i <br>";
    }
?>
```

The image shows a PHP code snippet for a for loop. Three red arrows point to the three parts of the for loop: the initialization (\$i= 0), the condition (\$i<5), and the increment (\$i=\$i+1).

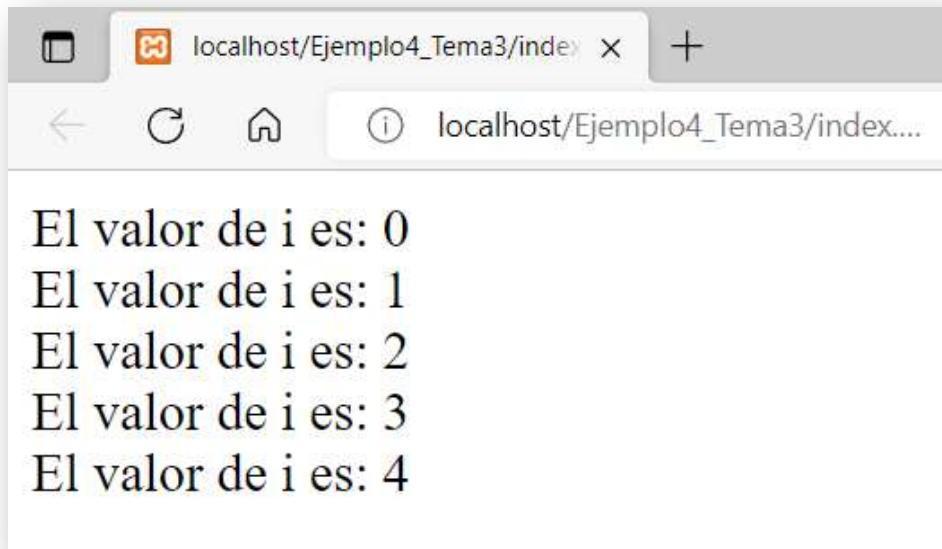
También valdrían las siguientes instrucciones:



```
<?php
    for($i= 0 ; $i<5 ; $i++)
    {
        echo "Valor de i es: $i <br>";
    }
?>
```

The image shows a PHP code snippet for a for loop with a different increment (\$i++).

La salida es la que sigue:



2.2.2.- Bucle *while*



Manual:

<https://www.php.net/manual/es/control-structures.while.php>

El bucle *while* también sigue la sintaxis habitual:

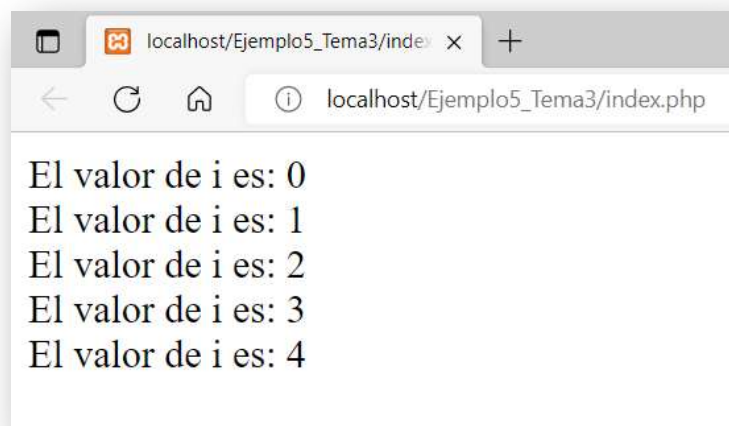
```
while (condición) {  
    instrucciones;  
}
```

Las instrucciones se ejecutan mientras se cumpla la condición. El *while* no cuenta con instrucciones de inicialización o iteración, pero se pueden añadir antes y al final del bucle, respectivamente.

El siguiente ejemplo tiene la misma salida que en el ejemplo con el bucle *for*:

```
<?php
    $i = 0;
    while ($i < 5)
    {
        echo "El valor de i es: $i <br>";
        $i = $i + 1;
    }
?>
```

Comprobamos la salida:



El valor de i es: 0
El valor de i es: 1
El valor de i es: 2
El valor de i es: 3
El valor de i es: 4

2.2.3.- Bucle do-while



Manual:

<https://www.php.net/manual/en/control-structures.do.while.php>

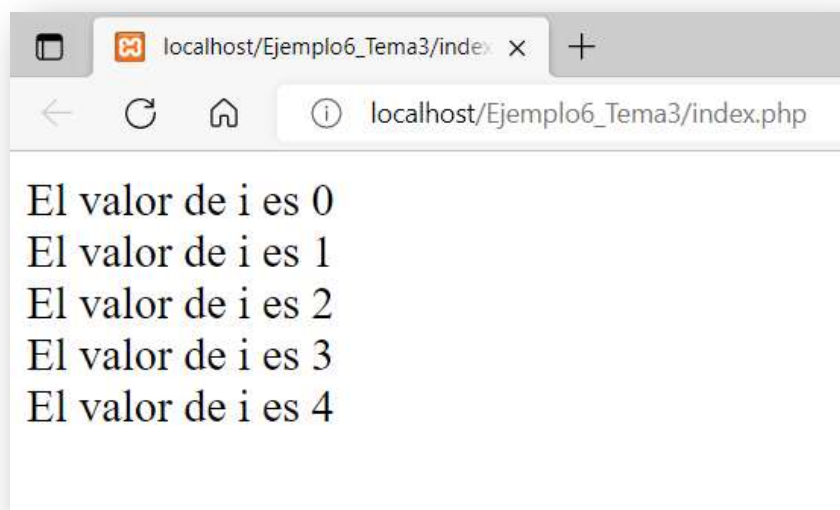
El bucle *do-while* es similar, pero la condición se evalúa después de ejecutar el bucle while.

```
do {
    instrucciones;
} while (condición);
```

Ejemplo:

```
<?php
    $i = 0;
    do{
        echo "El valor de i es $i <br>";
        $i = $i + 1;
    } while ($i < 5)
?>
```

Salida:



El bucle do-while se diferencia de los anteriores en que las instrucciones dentro del bucle se ejecutan por lo menos una vez.

- **Break y continue**



Manual:

<https://www.php.net/manual/es/control-structures.break.php>

Dentro de un bucle es posible usar las sentencias *break* y *continue*.

Break sirve para salir del bucle o *switch* en el que aparece. En PHP, esta sentencia admite un número, que representa el número de niveles de anidación de los que debe salir.

Ejemplo:

```
<?php
echo "Primer for anidado: <br>";
for ($i = 0; $i < 3; $i++) {
    for ($j = 0; $j < 3; $j++) {
        echo "i: $i j: $j <br>";
        if ($j == 1) {
            break; //es lo mismo que poner break 1
        }
    }
}
echo "Segundo for anidado: <br>";
for ($i = 0; $i < 3; $i++) {
    for ($j = 0; $j < 3; $j++) {
        echo "i: $i j: $j <br>";
        if ($j == 1){
            break 2;
        }
    }
}
```

Esto también se puede aplicar en una sentencia *switch*:

```
while ($i < 2) {
    switch ($i) {
        case 0:
            echo "Es un cero <br>";
            break 2;
        case 1:
            echo "Es un uno <br>";
            break;
    }
    $i++;
}
```

**Manual:**<https://www.php.net/manual/es/control-structures.continue.php>

Continue se utiliza dentro de las estructuras iterativas para saltar el resto de la iteración actual del bucle y continuar la ejecución en la evaluación de la condición, para luego comenzar la siguiente iteración.

Ejemplo:

```
<?php
    for ($i=0;$i<5;$i++)
    {
        if ($i==0)
        {
            continue;
        }
        echo "El valor de i es: $i <br>";
    }
?>
```

La salida es:

```
localhost/Ejemplo7_Tema3/index.php

El valor de i es: 1
El valor de i es: 2
El valor de i es: 3
El valor de i es: 4
```

2.2.4.- Otras estructuras de control

Es posible incluir otros ficheros utilizando las sentencias ***include*** y ***require***.

```
include "mifichero.php";  
require "mifichero.php";
```

Se diferencian solo en el tratamiento de errores.

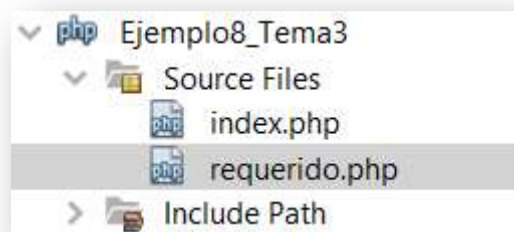
- Si no se encuentra el fichero especificado *include* genera un error de tipo **E_NOTICE**
- Si no se encuentra el fichero especificado *require* genera un error **E_FATAL** → implica el fin de la ejecución del *script*.

Si no hay error, en ambos casos el fichero especificado se ejecuta:

- Si el fichero es una librería con funciones, estas se declaran y quedan disponibles para el fichero que lo incluye.
- Si el fichero contiene sentencias fuera de una función, estas se ejecutan.

Ejemplo:

El árbol del proyecto es el siguiente:



El contenido del archivo "*index.php*" es el siguiente:

```
<?php  
    $a = "variable del programa principal";  
    require "requerido.php";  
    $b = "otra variable del programa principal";  
    echo "En el script principal";  
?>
```

Y el contenido de “requerido.php” es:

```
<?php
echo "En el fichero requerido <br>";
echo $a;
echo $b;
```

El resultado de la ejecución es:





¡ATENCIÓN!

¿Qué ocurre? ¿Por qué ese E_NOTICE?

En el fichero requerido
variable del programa principal
Notice: Undefined variable: b in C:\xampp\htdocs\Ejemplo8_Tema3\requerido.php on line 4
En el script principal

- Las instrucciones del fichero requerido se ejecutan
- Las variables del fichero principal están disponibles en el requerido si se habían inicializado antes de requerirlo.
 - Esto ocurre con \$a, pero no con \$b, que se inicializa después y, por tanto, se muestra un mensaje de error.

- El **ámbito** de las variables del fichero principal **incluye los ficheros requeridos**.

También están disponibles las sentencias ***require_once*** e ***include_once***, son idénticas a las anteriores excepto que PHP verificará si el archivo ya ha sido incluido y si es así, no se incluye (require o include) de nuevo.

- **Return**



Manual:

<https://www.php.net/manual/es/function.return.php>

Para acabar con las estructuras de control, la sentencia *return* finaliza la ejecución del fichero cuando se encuentra fuera de una función.

En el siguiente ejemplo, el *echo* final no se llega a ejecutar, porque se ha ejecutado antes el *return*

```
<?php
    $a = 0;
    if ($a == 0){
        return;
    }
    echo "Después del if";
```

Si el *script* se estaba ejecutando mediante un *require* o *include*, la ejecución continúa en el fichero que lo incluyó.



Si se pasa como argumento un número entero, éste se devuelve como valor de retorno



Si se encuentra dentro de una función, finaliza la ejecución de esta y la función devuelve el argumento del *return*.

3.- Arrays

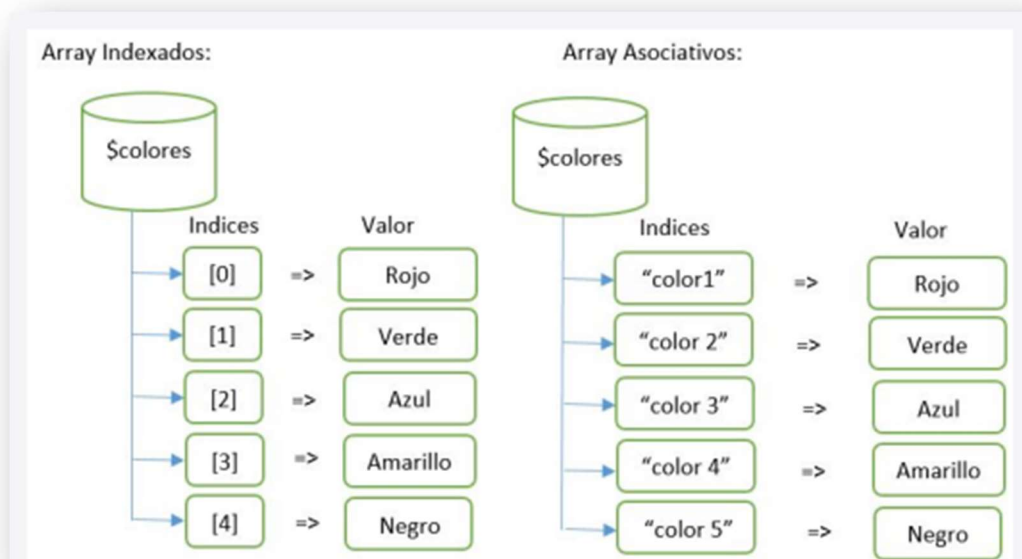


Manual:

<https://www.php.net/manual/en/language.types.array.php>

Los *arrays* en PHP son una estructura muy flexible y potente. Unifica en un solo tipo lo que en otros lenguajes se consigue con *arrays* básicos, vectores, listas o diccionarios.

Tenemos dos tipos de array en PHP:



Los elementos de un *array* se identifican por:

- una clave → que puede ser un entero o una cadena



Los elementos guardan un orden dentro del *array*. Este orden está determinado por el orden de los elementos al declarar el *array* o al añadir nuevos elementos.

Para declarar un *array* se puede utilizar la función ***array()***:

Ejemplo de ***array asociativo***:

```
$var = array(  
    clave1 => valor1,  
    ...  
    claven => valorn  
);
```

También es posible utilizar la notación de corchetes:

```
$arr = [  
    clave1 => valor1,  
    ...  
    claven => valorn  
];
```

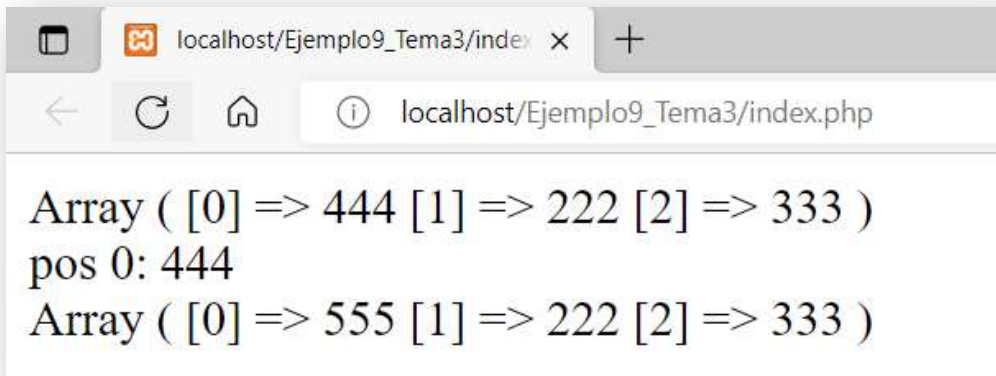
¿Cómo podemos acceder a cada elemento? Fácil:

`$arr[clave] = valor;`

Ejemplo:

```
<?php
$arr1= [
    0 => 444,
    1 => 222,
    2 => 333,
];
//Mostramos información sobre la variable $arr1
print_r($arr1);
//Se muestra la posición 0
echo "<br>". "pos 0: ".$arr1[0]. "<br>";
//Reasignamos el primer valor del array
$arr1[0]=555;
//Mostramos de nuevo información sobre el array
print_r($arr1);
echo "<br>";
```

La salida será:



```
Array ( [0] => 444 [1] => 222 [2] => 333 )
pos 0: 444
Array ( [0] => 555 [1] => 222 [2] => 333 )
```

Otro ejemplo más:

```
$arr2 = array (  
    "1111A" => "Pepe López",  
    "1112A" => "AntoniaVela",  
    "1113A" => "LuisMorales"  
);  
  
print_r($arr2);  
echo "<br>";  
  
$arr2["1113A"] = "Ana Tercero";  
  
print_r($arr2);
```

La correspondiente salida:

```
Array ( [1111A] => Pepe López [1112A] => AntoniaVela [1113A] => LuisMorales )  
Array ( [1111A] => Pepe López [1112A] => AntoniaVela [1113A] => Ana Tercero )
```

- También podemos definir un array utilizando la función "array()":

Ejemplo:

```
$cadenas = array("foo", "bar", "hello", "world");
```

```
$numeros = array(10, 20, 30, 40);
```

Podríamos crear un array vacío:

```
$numeros = array();
```

- Otra forma de definir un array:

```
$valores = [10, 20, 30, 40];
```



Acceso a los arrays:

\$array[índice]=valor;



Para recorrer un *array*, lo habitual es utilizar el bucle *foreach*.



Manual:

<https://www.php.net/manual/es/control-structures.foreach.php>

```
foreach($arr as $valor){  
    ...  
}
```

El cuerpo del bucle se ejecuta una vez por cada elemento del *array* **\$arr**. En cada iteración del bucle **\$valor** almacena el elemento correspondiente.

También se puede utilizar especificando una variable para la clave,

```
foreach($arr as $clave => $valor){ ... }
```

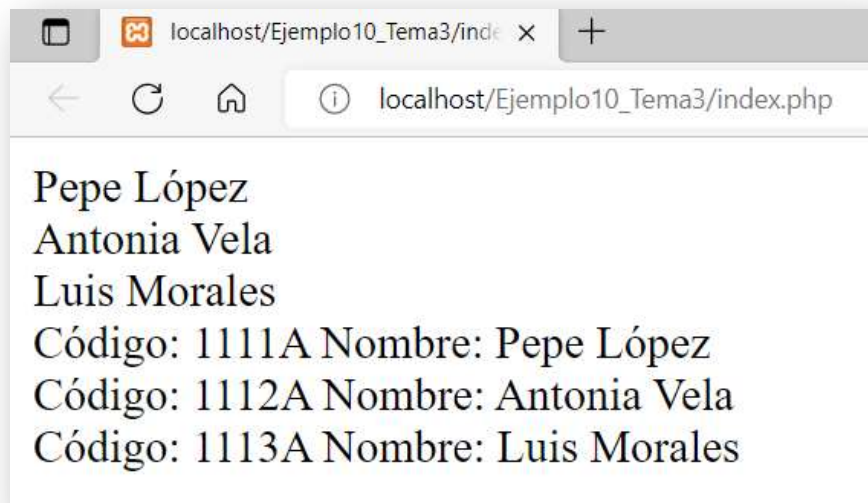
Ejemplo:

```
<?php
    $arr2 = array (
        "1111A" => "Pepe López",
        "1112A" => "Antonia Vela",
        "1113A" => "Luis Morales"
    );

    //Recorremos los arrays
    foreach ($arr2 as $nombre){
        echo "$nombre <br>";
    }

    foreach ($arr2 as $codigo => $nombre){
        echo "Código: $codigo Nombre: $nombre <br>";
    }
}
```

La salida en el navegador:



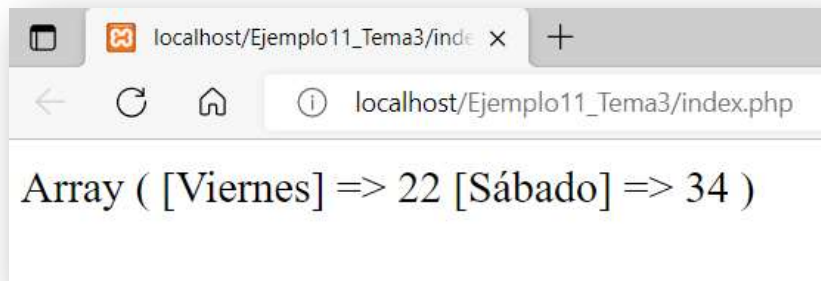
Si se pretende modificar el contenido del array al recorrerlo con un bucle *foreach*, hay que utilizar una referencia al declarar las variables del bucle.

Ejemplo. **No modificamos el array:**

```
<?php
$arr1 = array(
    "Viernes" => 22,
    "Sábado" => 34
);

/*No modificamos el array*/
foreach ($arr1 as $cantidad){
    $cantidad = $cantidad * 2;
}
print_r($arr1);
echo "<br>";
```

La salida:



localhost/Ejemplo11_Tema3/index.php

Array ([Viernes] => 22 [Sábado] => 34)

Vamos con el otro ejemplo, donde **SÍ se modifica el array:**

```
/*SI modificamos el array*/
foreach ($arr1 as &$cantidad){
    $cantidad = $cantidad * 2;
}

print_r($arr1);
```

Comprobemos la salida a ver qué ocurre:

```
Array ( [Viernes] => 44 [Sábado] => 68 )
```



También es posible declarar un *array* sin usar claves. En ese caso, se asignan como claves por defecto enteros consecutivos, empezando por 0.

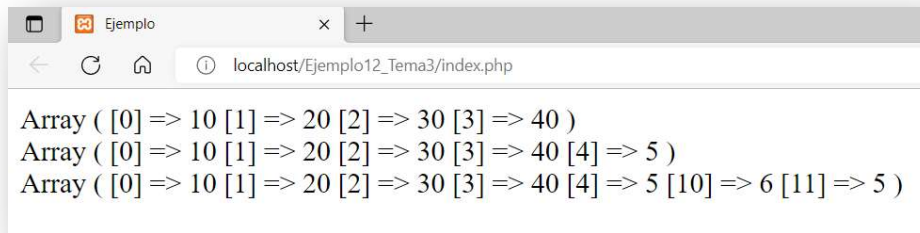


Si se añade un nuevo elemento sin especificar clave, la clave será el número siguiente a la mayor clave entera presente en el *array*.

Ejemplo:

```
<?php
    $arr1 = array(10, 20, 30, 40);
    print_r($arr1);
    echo "<br>";
    $arr1[] = 5;
    print_r($arr1);
    echo "<br>";
    $arr1[10] = 6;
    $arr1[] = 5;
    print_r($arr1);
    echo "<br>";
```

La salida es la siguiente:

A screenshot of a web browser window. The address bar shows 'localhost/Ejemplo12_Tema3/index.php'. The page content displays three lines of PHP array output:

```
Array ( [0] => 10 [1] => 20 [2] => 30 [3] => 40 )  
Array ( [0] => 10 [1] => 20 [2] => 30 [3] => 40 [4] => 5 )  
Array ( [0] => 10 [1] => 20 [2] => 30 [3] => 40 [4] => 5 [10] => 6 [11] => 5 )
```

3.1.- Operadores y arrays

3.1.1.- Operador Igual y operador Idéntico

Algunos operadores también están disponibles para los *arrays*, pero su significado no es exactamente el mismo.



Por ejemplo, el operador “Idéntico” solo será verdadero si los dos *arrays* tienen todos los elementos iguales, tanto en clave como en valor y además en el mismo orden.



Para el operador “Igual”, el orden de las claves no importa.

Ejemplos:

```
<?php
//Atención al tipo de variable y su valor
$arr1 = array(
    1 => "3000",
    2 => "4000"
);
$arr2 = array(
    1 => 3000,
    2 => 4000
);
$arr3 = array(
    1 => "4000",
    2 => "3000"
);
```

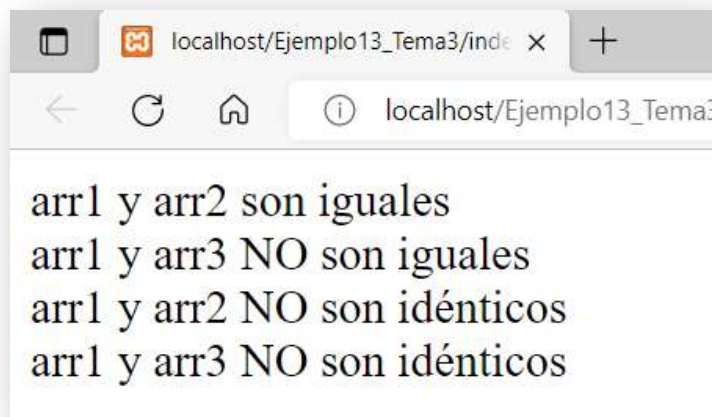
```
//Comparamos con == arr1 y arr2
if($arr1 == $arr2){
    echo "arr1 y arr2 son iguales <br>";
}
else{
    echo "arr1 y arr2 NO son iguales <br>";
}

//Comparamos con == arr1 y arr3
if($arr1 == $arr3){
    echo "arr1 y arr3 son iguales <br>";
}
else{
    echo "arr1 y arr3 NO son iguales <br>";
}
```

```
//Comparamos con === arr1 y arr2
if($arr1 === $arr2){
    echo "arr1 y arr2 son iguales <br>";
}
else{
    echo "arr1 y arr2 NO son iguales <br>";
}

//Comparamos con === arr1 y arr3
if($arr1 === $arr3){
    echo "arr1 y arr3 son iguales <br>";
}
else{
    echo "arr1 y arr3 NO son iguales <br>";
}
```

La salida será la siguiente:



3.1.1.- Operador +

El operador “+” utilizado con dos *arrays* devuelve su unión. El *array* resultante contendrá primero los elementos del primer *array* (que es el que aparece a la izquierda del operador) y a continuación los del segundo, pero sin repetir claves.



Si hay elementos con la misma clave en los dos *arrays*, el del segundo no se añade al resultado.

Ejemplo:

```
<?php
//Creamos el array 1
$arr1 = array (
    10 => "3000",
    20 => "4000",
    30 => "6000"
);
print_r($arr1);
echo "<br>";
//Creamos el array 2
$arr2 = array (
    10 => "8000",
    15 => "6000",
    20 => "4000"
);
print_r($arr2);
echo "<br>";
```

```
//Utilizamos el operador +
$arr3=$arr1 + $arr2;
print_r($arr3);
```

La salida es:

```
localhost/Ejemplo14_Tema3/index.php
Array ( [10] => 3000 [20] => 4000 [30] => 6000 )
Array ( [10] => 8000 [15] => 6000 [20] => 4000 )
Array ( [10] => 3000 [20] => 4000 [30] => 6000 [15] => 6000 )
```

A continuación, se muestra un resumen:

Operador	Descripción
<code>\$a1 === \$a2</code>	Idéntico. Verdadero si los dos <i>arrays</i> tienen las claves y valores iguales, en el mismo orden y del mismo tipo.
<code>\$a1 !== \$a2</code>	No idéntico.
<code>\$a1 == \$a2</code>	Igual. Verdadero si los dos <i>arrays</i> tienen las claves y valores iguales.
<code>\$a1 != \$a2</code> , <code>\$a1 <> \$a2</code>	No igual.
<code>\$a1 + \$a2</code>	Unión. Devuelve un <i>array</i> con los elementos de ambos.

Tabla resumen:

Forma de Definición	Ejemplo
Usar la función <code>array()</code>	<code>\$frutas = array("manzana", "plátano", "uva");</code>
Sintaxis corta de corchetes <code>[]</code> (usada a partir de PHP5.4)	<code>\$frutas = ["manzana", "plátano", "uva"];</code>
Definición de un <i>array</i> asociativo	<code>\$persona = ["nombre" => "Juan", "edad" => 30, "ciudad" => "México"];</code>
Usar la función <code>range()</code>	<code>\$numeros = range(1, 5);</code>
Convertir una cadena en un <i>array</i>	<code>\$cadena = "Hola"; \$letras = str_split(\$cadena);</code>

4.- Funciones y librerías



Manual:

<https://www.php.net/manual/es/language.functions.php>

Una **función** es un conjunto de instrucciones que realiza una tarea concreta.

- Las funciones pueden recibir argumentos, que son los datos que necesitan para llevar a cabo su cometido.
- Las funciones pueden devolver o no un valor.



A diferencia de lo que ocurre en otros lenguajes, en PHP no es necesario declarar el tipo de dato que se devuelve en una función y, de hecho, puede devolver tipos de datos diferentes según el caso.

Por ejemplo, en PHP hay muchas funciones que devuelven:

- FALSE → en caso de error
- Otro tipo de datos (entero, cadena, etc...) → si no hay error.

4.1.- Funciones predefinidas

PHP cuenta con una gran cantidad de funciones predefinidas para las tareas más habituales.

Como en PHP las variables no se declaran explícitamente, hay una serie de funciones que permiten conocer el estado y el tipo de dato de una variable. Las más importantes son:

- **is_null(\$var)** → Devuelve TRUE si \$var es NULL, FALSE en otro caso.
- **isset(\$var)** → Devuelve TRUE si \$var ha sido inicializada y su valor no es NULL, FALSE en otro caso.
- **unset(\$var)** → Elimina la variable. Ya no contará como inicializada.

- **empty(\$var)** → Devuelve TRUE si \$var no ha sido inicializada o su valor es FALSE, FALSE en otro caso.
- **is_int(\$var), is_float(\$var), is_bool(\$var), is_array(\$var)** → Devuelven TRUE si \$var es entero, float, booleano o array respectivamente, y FALSE en otro caso.
- **print_r(\$var)** y **var_dump(\$var)** → Muestran información sobre \$var.

Existen más funciones predefinidas, que iremos utilizando según hagamos ejercicios.

De nuevo, se muestran unas tablas resumen que ayudan bastante a familiarizarse con dichas funciones:

Funciones de variables	
isset(\$var)	TRUE si la variable está inicializada y no es NULL
is_null(\$var)	TRUE si la variable es NULL
empty(\$var)	TRUE si la variable no está inicializada o su valor es FALSE
is_int(\$var), is_float(\$var), is_bool(\$var), is_array(\$var)	Para comprobar el tipo de dato de \$var
intval(\$var), floatval(\$var), boolvar(\$var), strval(\$var)	Para obtener el valor de \$var como otro tipo de dato

Funciones de cadenas	
strlen(\$cad)	Devuelve la longitud de \$cad
explode(\$cad, \$token)	Parte una cadena utilizando \$token como separador. Devuelve un array de cadenas
implode(\$token, \$array)	Crea una cadena larga a partir de un array de cadenas, entre cadena y cadena se introduce \$token
strcmp(\$cad1, \$cad2)	Compara las dos cadenas. Devuelve 0 si son iguales, -1 si \$cad1 es menor y 1 si \$cad1 es mayor
strtolower(\$cad), strtoupper(\$cad)	Devuelven \$cad en mayúsculas o minúsculas, respectivamente
str(\$cad1, \$cad2)	Busca la primera ocurrencia de \$cad2 en \$cad1. Si no aparece devuelve FALSE, si aparece devuelve \$cad1 desde donde comienza la ocurrencia

Funciones de arrays	
ksort(\$arr), krsort(\$arr)	Ordena el array por clave en orden ascendente o descendente
sort(\$arr), rsort(\$arr)	Ordena el array por valor en orden ascendente o descendente
array_values(\$arr)	Devuelve los valores de \$arr
array_keys(\$arr)	Devuelve las claves de \$arr
array_key_exists(\$arr, \$cla)	Devuelve verdadero si algún elemento de \$arr tiene clave \$cla
count(\$arr)	Devuelve el número de elementos del array

4.2.- Funciones definidas por el usuario

La sintaxis para definir una función es:

```
function nombre(argumentos){
    <instrucciones>
}
```

La función **puede tener o no argumentos**. Se declaran en la cabecera de la función, entre los paréntesis, separados por comas.



Los argumentos son variables locales a la función y sólo existen mientras ésta se esté ejecutando.

En PHP no hace falta declarar el tipo de dato que devuelve la función. De hecho, ya se expresó más arriba, que **es posible que una función devuelva tipos de datos diferentes según el caso**.

Por ejemplo:

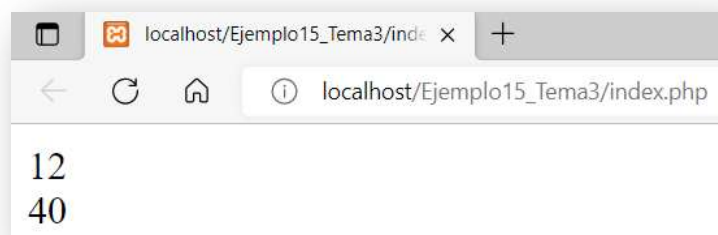
```
<?php
//Definimos la función
function suma ($a, $b){
    return $a+$b;
}

echo suma(4,8). '<br>';

$var1 = 35;
$var2 = 5;

$var3=suma($var1,$var2);
echo $var3. '<br>';
```

La salida es la siguiente:



Es posible especificar valores por defecto para los argumentos. Si al llamar a la función no se usa el argumento, se toma el valor por defecto.

Ejemplo:

```
<?php
function saludar($nombre='Daddy Yankee'){
    echo "Hola $nombre <br>";
}

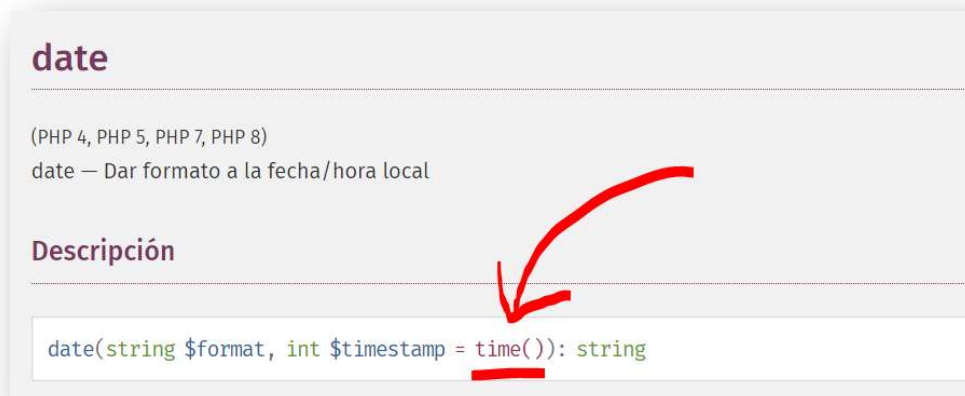
saludar();
saludar("Maluma Baby");
```

Salida:



¡Recordemos algo! Vamos a mirar la función `date()` en el manual de PHP, a ver si vemos algo relacionado con los valores por defecto en una función.

¿Qué ocurre cuando buscamos la función `date()` en la página oficial de PHP y queremos ver qué parámetros hay que pasarle?



En este caso, el segundo parámetro es “opcional”, si solo pasamos a la función un parámetro, por defecto “date” utilizará otra función llamada “time” para realizar las operaciones que necesite realizar.

4.3.- Paso de parámetros por copia y por referencia

En PHP los **argumentos** se pasan **por copia**. Esto quiere decir que, cuando se llama a una función con una variable como argumento, se crea una variable local en la función y se copia el valor del argumento en dicha variable local.



Por tanto, si la función modifica el argumento, estos cambios no tendrán efecto en la variable original.

Para solucionar esto hay dos opciones:

- **Devolver el nuevo valor y reasignar** → como en la función *duplicar()* del ejemplo siguiente.
- **Utilizar una referencia** → Como en la función *duplicar2()* del ejemplo.

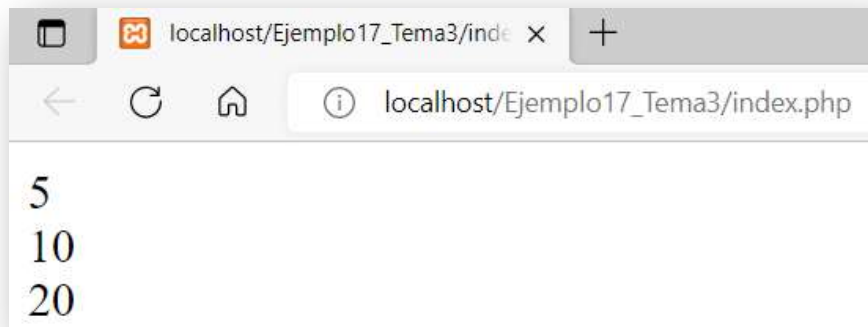
```
<?php
//Se definen las funciones
function duplicarMal($a){
    $a = $a *2;
}

function duplicar($a){
    return $a *2;
}

function duplicar2(&$a){
    $a = $a *2;
}
```

```
//Llamamos a las funciones  
$var1 = 5;  
  
duplicarMal($var1);  
echo "$var1 <br>";  
  
$var1=duplicar($var1);  
echo "$var1 <br>";  
  
duplicar2($var1);  
echo "$var1 <br>";
```

La salida será, por tanto:



4.4.- Funciones como argumentos

En PHP es posible pasar funciones como argumentos a otras funciones. Es una característica avanzada que se utiliza entre otras cosas para las funciones de *callback*.



Solo hay que pasar el nombre de la función entre comillas como argumento. La función que lo recibe podrá utilizarla si le pasa los argumentos adecuados.

Ejemplo:

```
<?php

//Definición de las funciones
function calculador($operacion,$num1,$num2){
    $result=$operacion($num1,$num2);
    return $result;
}

function sumar($a,$b){
    return $a+$b;
}

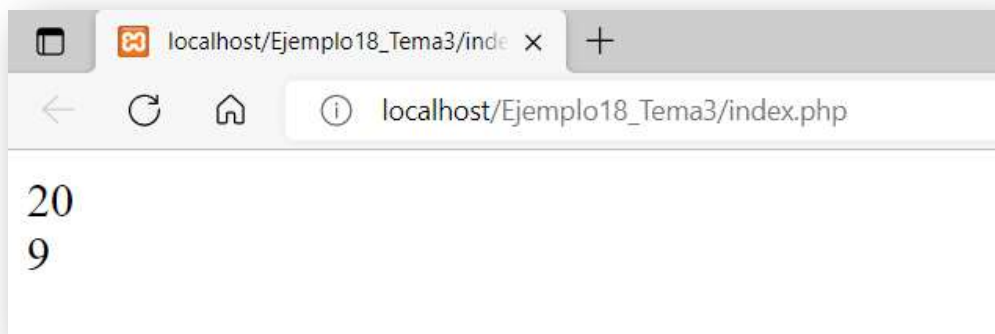
function multiplicar($a,$b){
    return $a*$b;
}
```

```
//Comprobamos el resultado
$a = 4;
$b = 5;

$r1=calculador("multiplicar", $a,$b);
echo "$r1 <br>";

$r2=calculador("sumar", $a,$b);
echo "$r2 <br>";
```

El resultado que nos muestra el navegador es:



Si quieres ampliar información sobre este tipo de paso de parámetro para las funciones, consulta este enlace → <https://blogprog.gonzalopez.es/articulos/pasar-una-funcion-como-parametro-de-otra-funcion-en-php.html>

Ámbitos de la variable en una función



Probemos los siguientes casos a ver qué ocurre:

CASO 1

```
$num = 22;  
  
function prueba() {  
    echo $num;  
}  
  
prueba();
```

CASO 2

```
function prueba() {  
    $num=22;  
    echo $num;  
}  
  
prueba();
```

CASO 3

```
$num = 22;  
  
function prueba() {  
    global $num;  
    echo $num;  
}  
  
prueba();
```



5.- Excepciones y errores

El sistema de control de errores de PHP ha ido evolucionando a lo largo de las versiones. En el sistema básico se generan errores de diferentes tipos, representados por un número.

Por otro lado, desde PHP5 hay un sistema de excepciones similar al de Java y otros lenguajes utilizando bloques **try** y **catch/finally**. Finalmente, en PHP7 aparecieron las excepciones de clases **Error**.

5.1.- Errores

En el sistema básico, ante determinadas condiciones (por ejemplo, utilizar una variable no inicializada) **PHP genera un error**.

Hay diferentes tipos de errores, cada uno asociado con un número y constante predefinida.



Puede controlarse el comportamiento de PHP ante los errores mediante tres directivas del fichero *php.ini*:

- **error_reporting** → indica qué errores deben reportarse. Lo normal es utilizar E_ALL, es decir, todos.
- **display_error** → señala si los mensajes de error deben aparecer en la salida del script. Esta opción es apropiada durante el desarrollo, pero no en producción
- **log_error** → Indica si los mensajes de error deben almacenarse en un fichero. Es especialmente útil en producción, cuando no se muestran errores de salida.
- **error_log** → si la directiva anterior está activada, es la ruta en la que se guardan los mensajes de error.



El valor de la directiva *error_reporting* es un número, pero para especificarlo lo habitual es utilizar las constantes predefinidas.

Tipos de error:

Código	Constante	Descripción
1	E_ERROR	Error fatal en tiempo de ejecución. La ejecución del <i>script</i> se detiene
2	E_WARNING	Advertencia en tiempo de ejecución. El <i>script</i> no se detiene
4	E_PARSE	Error de sintaxis al compilar
8	E_NOTICE	Notificación. Puede indicar error o no
16	E_CORE_ERROR	Error fatal al iniciar PHP
32	E_CORE_WARNING	Advertencia al iniciar PHP
64	E_COMPILE_ERROR	Error fatal al compilar
128	E_COMPILE_WARNING	Advertencia fatal al compilar
256	E_USER_ERROR	Error generado por el usuario
512	E_USER_WARNING	Advertencia generada por el usuario
1024	E_USER_NOTICE	Notificación generada por el usuario
2048	E_STRICT	Sugerencias para mejorar la portabilidad
4096	E_RECOVERABLE_ERROR	Error fatal capturable
8192	E_DEPRECATED	Advertencia de código obsoleto
16384	E_USER_DEPRECATED	Como la anterior, generada por el usuario
32767	E_ALL	Todos los errores

5.2.- Excepciones

Otra opción para indicar un error es lanzar una excepción. Para controlar las excepciones se utilizan bloques ***try/catch/finally***, como en Java.

Cuando se lanza una excepción y no es capturada por un bloque *catch*, la ejecución del programa se detiene.

Se puede añadir un bloque ***finally***, que se ejecuta después del ***try/catch***, haya habido excepción o no.

```
try{
    instrucciones;
}catch(Exception e){
    instrucciones;
}finally{
    instrucciones;
}
```

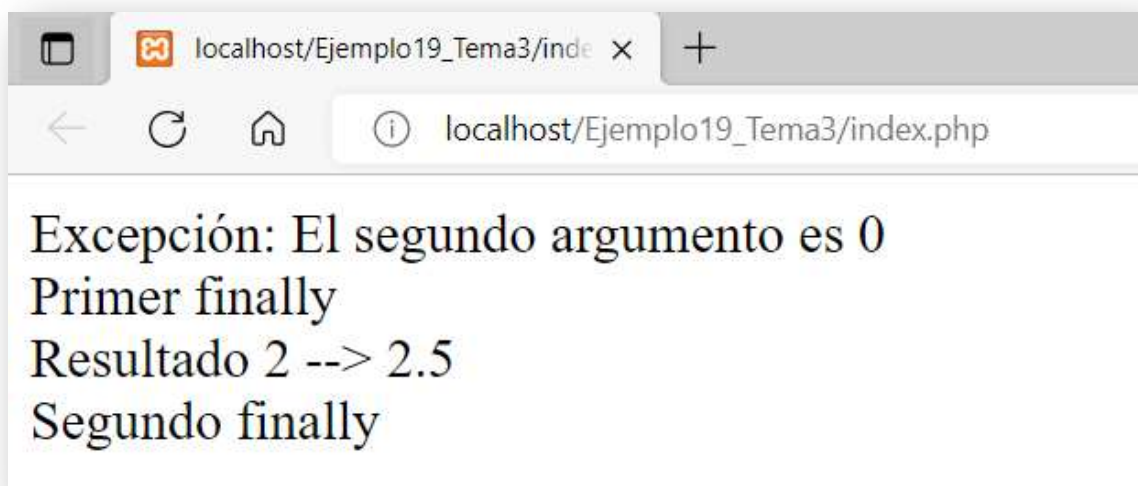
Ejemplo:

```
<?php
//Se define la función suma
function dividir($a,$b){
    //Se crea la excepción
    if($b==0){
        throw new Exception ('El segundo argumento es 0');
    }
    return $a/$b;
}
```

```
//Intentamos dividir por 0
try{
    $result1=dividir(5,0);
    echo "Resultado 1 --> $result1"."<br>";
} catch (Exception $ex) {
    echo "Excepción: ".$ex->getMessage()."<br>";
}finally{
    echo "Primer finally <br>";
}
```

```
//La división se hará sin problema
try{
    $result2=dividir(5,2);
    echo "Resultado 2 --> $result2". "<br>";
} catch (Exception $ex) {
    echo "Excepción: ".$ex->getMessage(). "<br>";
}finally{
    echo "Segundo finally";
}
```

¿Qué mostrará el navegador en esta ocasión?



5.3.- Excepciones error

En PHP7 aparecieron las excepciones de tipo Error. No heredan de la clase **Exception**, así que para capturarlas hay que usar:

```
catch (Error $e){
    ...
}
```

O, alternatively, the class **Throwable**, of which both *Error* and *Exception*

```
catch (Throwable $e){  
    ...  
}
```

Excepciones de error predefinidas:

Nombre	Descripción	Hereda de
Error	Clase base para las excepciones Error	
ArithmeticError	Error en operaciones matemáticas. Hereda del anterior	Error
DivisionByZeroError	Intento de división por cero. Hereda del anterior	ArithmeticError
AssertionError	Ocurre cuando falla una llamada a assert()	Error
ParseError	Error al compilar	Error
TypeError	Ocurre cuando una expresión no tiene el tipo de dato que se espera	Error
ArgumentCountError	Ocurre al llamar a una función con menos argumentos de los necesarios. Hereda del anterior	TypeError

6.- Formularios

Los formularios HTML son la forma más habitual de **enviar datos a un servidor**. Permiten que el usuario rellene varios campos mediante diferentes tipos de controles (campos de texto, botones de radio...) y lo envíe al servidor al pulsar un botón.

El servidor procesa los datos y genera una respuesta.

Un formulario sencillo de *login* en HTML se escribiría así:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Formulario de login</title>
    <meta charset = "UTF-8">
  </head>
  <body>
    <form action = "login_basico.php" method = "POST">
      <input name = "usuario" type = "text">
      <input name = "clave" type = "password">
      <input type = "submit">
    </form>
  </body>
</html>
```

- El atributo **action** del formulario especifica la ruta del **script** al que se enviará el formulario para que lo procese.



Si se usa una ruta relativa, se toma como referencia la localización en el servidor del fichero que contenga el formulario.

- El atributo **method** especifica el método HTTP, que se usará en la petición.



En los formularios, lo habitual es utilizar POST, pero también es posible usar GET.

Si se utiliza GET, los parámetros NO aparecen en la URL

- Dentro del elemento **form** se introducen los campos, que tendrá que rellenar el usuario, y los botones de envío. También para limpiar los campos.

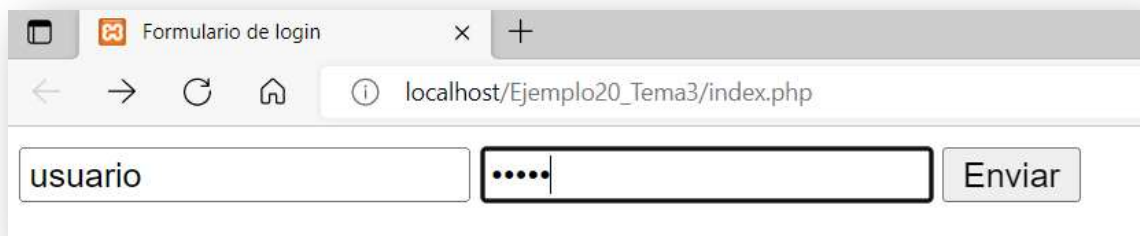


Para los campos que se envían hay que usar el atributo name, que sirve para identificarlo dentro del *script*.

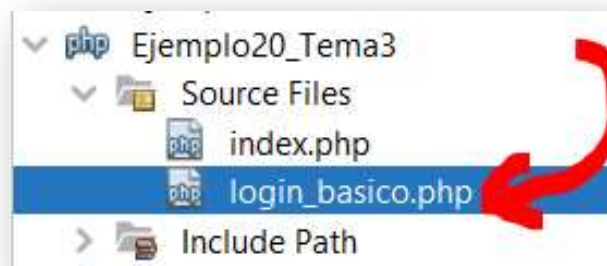
El envío se produce al **pulsar el botón**.

En el *script* al que se envía el formulario, los parámetros están disponibles en el *array* superglobal `$_POST`. La clave de cada argumento dentro del *array* es el atributo **name** del elemento correspondiente en el formulario.

Vamos a ejecutar la prueba, a ver qué ocurre:



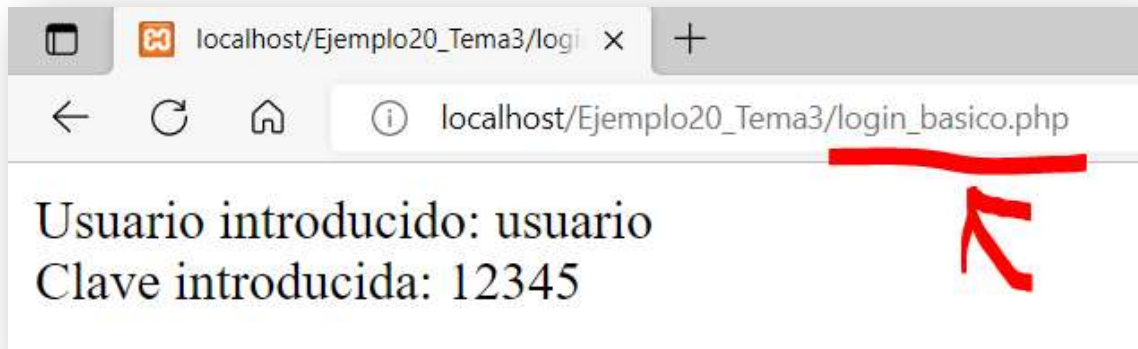
Hemos creado otro archivo PHP llamado "login_basico.php" que es donde nos llevará la aplicación después de hacer clic en el botón.



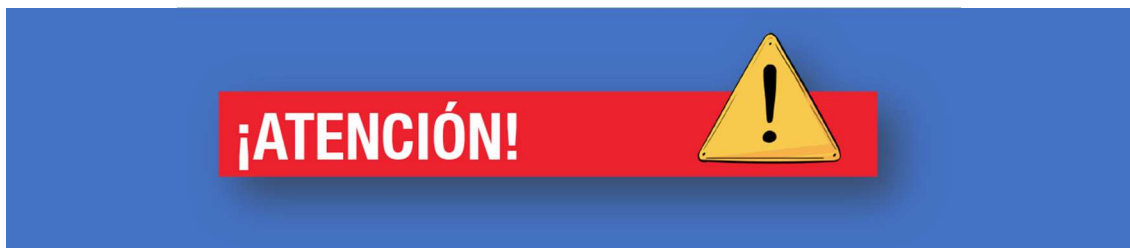
En dicho archivo, se ha creado el siguiente script:

```
<?php
echo "Usuario introducido: ".$_POST["usuario"]."<br>";
echo "Clave introducida: ".$_POST["clave"];
```

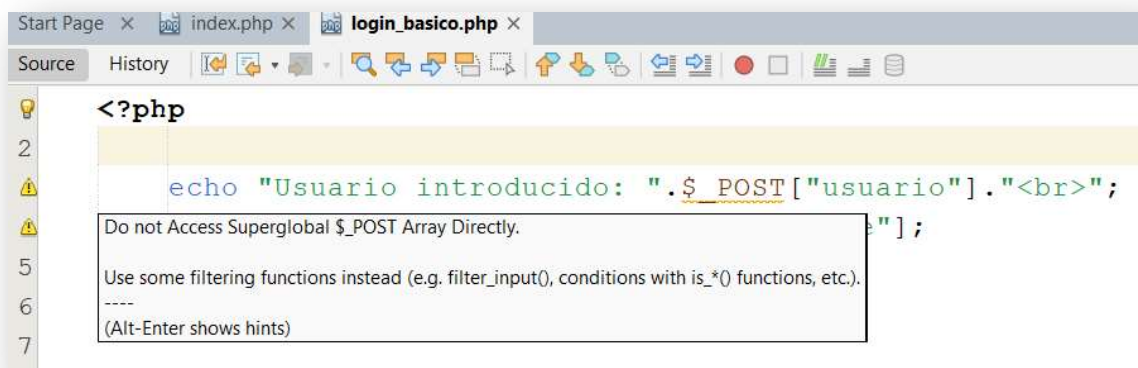
Veamos la salida:



Como se puede observar, al hacer clic en el botón “enviar” que hay en el archivo llamado “index.php”, la página nos redirige al otro archivo “login_basico.php”, mostrándonos el valor de la información que hemos enviado por el método POST.



Si nos fijamos en el script “login_basico.php”, nos aparecen dos *Warnings*:



Este warning es debido a que en las nuevas versiones de PHP no se recomienda el uso directo de las variables superglobales como

las usadas en los formularios y métodos POST y GET. En su lugar se recomienda filtrar la información recibida, esto es **por seguridad**.



Manual:

<https://www.php.net/manual/en/function.filter-input.php>

La solución es la siguiente:

```
<?php

if(filter_input_array(INPUT_POST)){
    echo "Usuario introducido: ".filter_input(INPUT_POST, 'usuario')."<br>";
}
```

6.1.- Formulario de *login*



En general, un formulario de *login* se encarga de comprobar los datos introducidos y, según sean correctos o no, da acceso al sistema al usuario o muestra un mensaje de error.

En *login_basico.php*, programamos lo siguiente:

```
<?php

//Comprobamos a mano si es "usuario" el valor de usuario
if(filter_input_array(INPUT_POST)){
    echo "Usuario introducido: ".filter_input(INPUT_POST,'usuario')."<br>";
    echo "Clave introducida: ".filter_input(INPUT_POST,'clave');

    if(filter_input(INPUT_POST,'usuario')== "usuario" &&
        filter_input(INPUT_POST,'clave')== "12345")
    {
        header("Location:bienvenido.html");
    }
    else{
        header("Location:error.html");
    }
}
```

Ocurrirán dos cosas:

- 1) Si las credenciales coinciden con “usuario” y “12345” (para la clave) → PHP nos redirigirá a una página llamada “bienvenido.html”



- 2) Si las credenciales no coinciden con “usuario” y “12345” (para la clave) → PHP nos redirigirá a una página de error (error.html)



Hay que enviar las cabeceras antes de empezar con el cuerpo de la respuesta. Esto implica que hay que utilizar la función *header()* antes de que se empiece a escribir la salida. Si se intenta llamar a *header()* después de haber realizado un *echo*, se producirá un error.

6.2.- Formulario y procesamiento en un solo fichero

En ocasiones el formulario HTML y el bloque PHP que lo procesa se integran en un solo fichero, en el que hay que distinguir dos casos:

- Cuando se accede al formulario para rellenarlo
- Cuando se envía para procesarlo.

Cuando se accede a la página usando el método GET, es decir, introduciendo la dirección en el navegador. Al seguir un vínculo o como resultado de una redirección con ***header(Location:)***, se muestra el formulario.

En cambio, si se accede mediante POST quiere decir que el cliente está enviando el formulario.

Se puede diferenciar entre los dos métodos de acceso consultando:

`$_SERVER["REQUEST_METHOD"]`

Ejemplo:

```
<?php

/*Si va bien, redirige a principal.php,
 * si va mal mensaje de error
 */
if($_SERVER["REQUEST_METHOD"]=="POST")
{
    if($_POST["usuario"]=="usuario" && $_POST["clave"]=="12345")
    {
        header("Location:principal.php");
    }else{
        $err=true;
    }
}

?>
```

```
<html lang="es">
<head>
<meta charset="UTF-8">
<title>Formulario de login</title>
</head>
<body>
<?php if(isset($err)){
    echo "<p> Revise usuario y contraseña</p>";
}??
<form action="<?php echo htmlspecialchars($_SERVER["PHP_SELF"]);?>" method="POST">
    <label for="usuario">Usuario</label>
    <input value="<?php if(isset($usuario)) echo $usuario;?>"
        id="usuario" name="usuario" type="text">
    <label for="clave">Clave</label>
    <input name="clave" type="password">
    <input type="submit">
</form>
</body>
</html>
```

Cuando el formulario llama al mismo fichero, se recomienda usar:

```
action="<?php echo htmlspecialchars($_SERVER["PHP_SELF"]);?>"
```



La variable *superglobal* `$_SERVER["PHP_SELF"]` contiene el nombre del fichero y la función `htmlspecialchars()` sirve para filtrar los caracteres por seguridad

El bloque inicial de PHP (con la comprobación de usuario y contraseña) se ejecuta solo cuando se accede por el método POST, es decir, al **enviar el formulario**.

7.- Glosario de términos

Array. Es un tipo de dato compuesto, habitual en los lenguajes de programación. Aunque los detalles varían entre lenguajes, en general, se parecen a vectores o listas ordenadas.

Bucle. Estructura de programación que permite repetir instrucciones.

Estructura condicional. Posibilita ejecutar o no una instrucción según se cumpla una condición

Función. Conjunto de instrucciones que realiza una tarea concreta.

Librería. Las funciones relacionadas entre sí se agrupan en librerías o bibliotecas.

Script. Programa sencillo, habitualmente ejecutado por un intérprete en lugar de ser compilado.

Variable. Posición en la memoria del ordenador identificada por un nombre. La variable almacena datos. Estos datos son el valor de la variable.

8.- Referencias bibliográficas

- ❖ Sanz, M. L. *Programación Web en el Entorno Servidor*. (MF0492_3). Grupo Editorial RA-MA.
- ❖ Ganzábal, Xabier. *Desarrollo web en entorno servidor*. Editorial Síntesis.
- ❖ PHP (20 de septiembre de 22). *Manual oficial de PHP*
<https://www.php.net/manual/es/index.php>